CS3021 Introduction to Data Structures and Intermediate Programming

Sorting Sequences

Objectives

- Describe the importance of *sorting* a collection of elements to allow for more efficient search.
- Learn how to perform fundamental sort algorithms: *selection sort* and *bubble sort*.
- Describe the *heap* data structure, and the associated *heapsort*.

Sorting for Search Performance

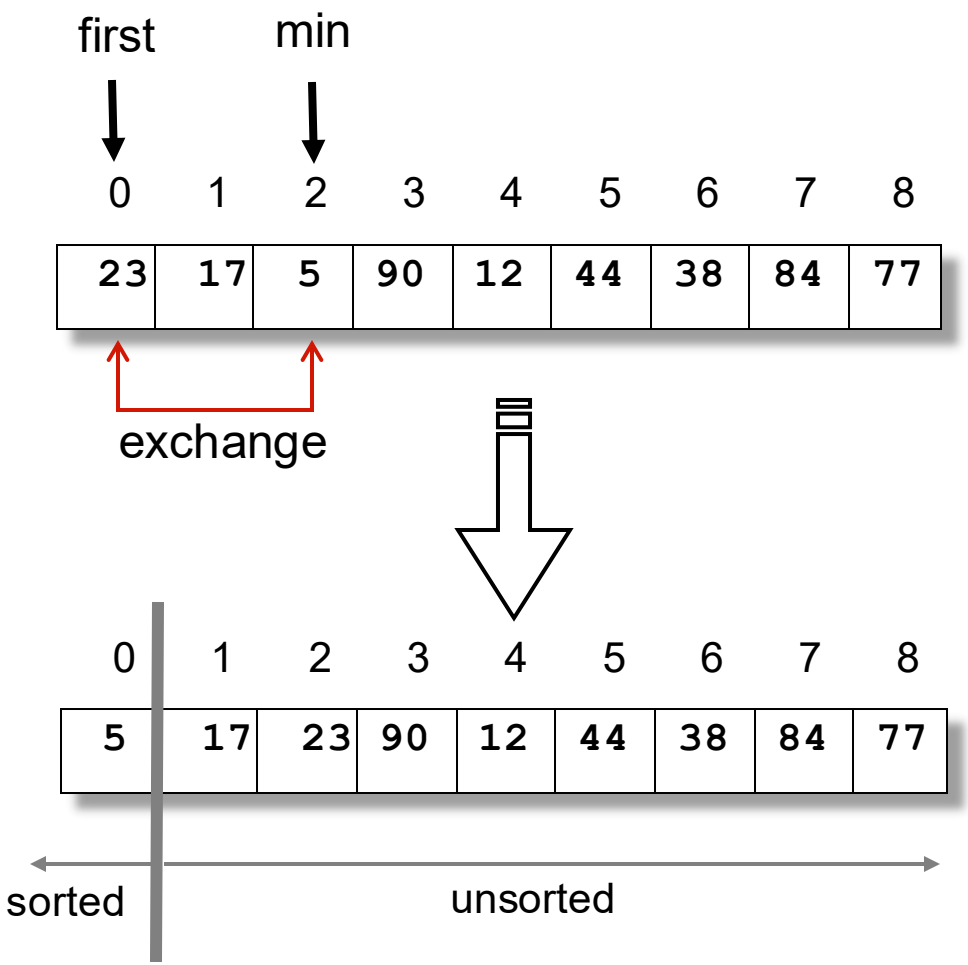
- Clearly a logarithmic performing search is better, but it relies on the data being sorted first.
- So, we need to consider how we can effectively and efficiently sort a sequence of data, to support better searching (among other functions).
- We will use an array of integers to present various sorting algorithms.

Selection Sort

1. Find the smallest element in the array.

2. Exchange that element with the one in the first position – the smallest element is now in the first position.

3. Repeat Step 1 and 2 with the array having one less element each pass (i.e., the smallest element is discarded from further processing).



This is the result of one pass

Selection Sort Passes

Pass #	0	1	2	3	4	5	6	7	8
1	5	17	23	90	12	44	38	84	77
2	5	12	23	90	17	44	38	84	77
3	5	12	17	90	23	44	38	84	77
•••									
7	5	12	17	23	38	44	77	84	90
8	5	12	17	23	38	44	77	84	90

sorted

Result AFTER one pass is completed

Selection Sort Routine

```
void selectionSort(int number[], int length)
{
    int startIndex, minIndex, temp;

    for (startIndex = 0; startIndex <= length-2; startIndex++){
        // each iteration of the for loop is one pass
        minIndex = startIndex;

        // find the smallest value in this pass at position minIndex
        for (i = startIndex+1; i <= length-1; i++) {
            if (number[i] < number[minIndex]) minIndex = i;
        }
        // swap number[startIndex] and number[minIndex]
        temp                = number[startIndex];
        number[startIndex] = number[minIndex];
        number[minIndex]    = temp;
    }
}
```

Selection Sort Performance

- We derive the total number of comparisons by counting the number of times the inner loop is executed.
- For each execution of the outer loop, the inner loop is executed length – start times.
- The variable length represents the size of the array. Replacing length with *N* (array size), the sum is derived as…

Start	Number of Comparisons (Length – Start)
0	length
1	length – 1
2	length – 2
. . .	. . .
length – 2	2

$$N + (N-1) + (N-2) + \cdots + 2$$

$$= \sum_{i=2}^{N} i = \left(\sum_{i=1}^{N} i\right) - 1 = \frac{N(N+1)}{2} - 1$$

$$= \frac{N^2 + N - 2}{2} \approx N^2$$

Bubble Sort

- With the *selection sort*, we make one exchange at the end of one pass.
- The *bubble sort* improves performance by making more than one exchange during each pass.
- By making multiple exchanges, we are able to move more elements toward their correct positions using the same number of comparisons as *selection sort* makes.
- The key idea of the bubble sort is to make *pairwise* comparisons, and exchange the positions of the pair being compared, if they are out of order.

One Pass of Bubble Sort

0	1	2	3	4	5	6	7	8
23	17	5	90	12	44	38	84	77

exchange

17	23	5	90	12	44	38	84	77

exchange

17	5	23	90	12	44	38	84	77

ok exchange

17	5	23	12	90	44	38	84	77

exchange

17	5	23	12	44	90	38	84	77

exchange

17	5	23	12	44	38	90	84	77

exchange

17	5	23	12	44	38	84	90	77

exchange

17	5	23	12	44	38	84	77	90

The largest value (90) is now at the end of the array, in its proper place.

Bubble Sort Routine

```
void bubbleSort(int number[], int length)
{
    int index, bottom, temp;
    bottom = length - 2;  // penultimate array element
    bool exchanged = true;

    while (exchanged)  {
        exchanged = false;
        for (index = 0; index <= bottom; index++) {
            if (number[index] > number[index+1]) {  // swap them
                temp = number[i];
                number[index] = number[index+1];
                number[index+1] = temp;

                exchanged = true; // swap was made
            }
        }
        bottom--;
    }
} // end bubbleSort
```

Bubble Sort Performance

- In the worst case, the outer *while*-loop is executed *N*-1 times.
- For each execution of the outer *while*-loop, the inner *for*-loop is executed bottom+1 times.
- The number of comparisons in each successive pass is *N*-1, *N*-2, …, 1. Summing these results in total number of comparisons.

$$(N-1)+(N-2)+\cdots+1$$

$$=\sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}$$

$$=\frac{N^2-N}{2} \cong N^2$$

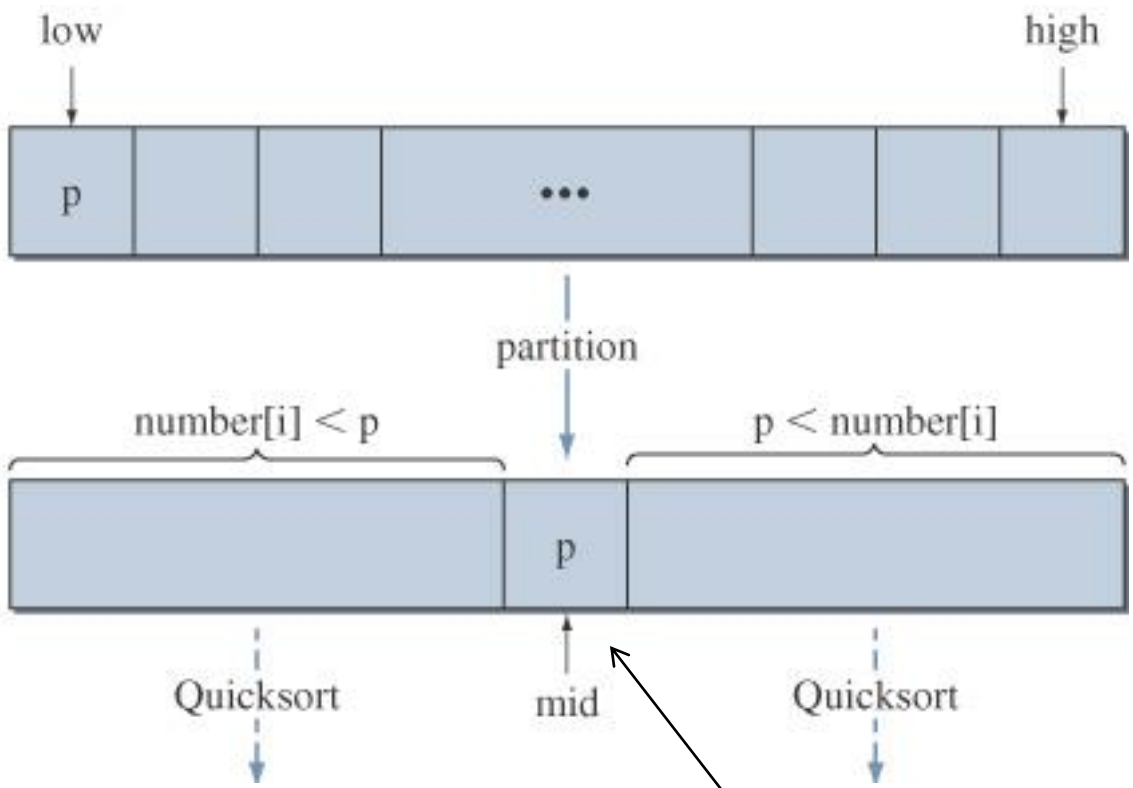
- So, *bubble* and *selection* sort performances are approximately equivalent. However, on average, *bubble* sort performs much better because it can finish sorting in fewer than *N*-1 passes... but we can still do better.

Quicksort

- A recursive "*divide-and-conquer*" sorting algorithm:
  - Partition array by first choosing a (random) *pivot* element from the array, and then...
  - Reorder array so that all elements before the *pivot* are less than the pivot, and all elements after are greater; now...
  - Recursively sort each array

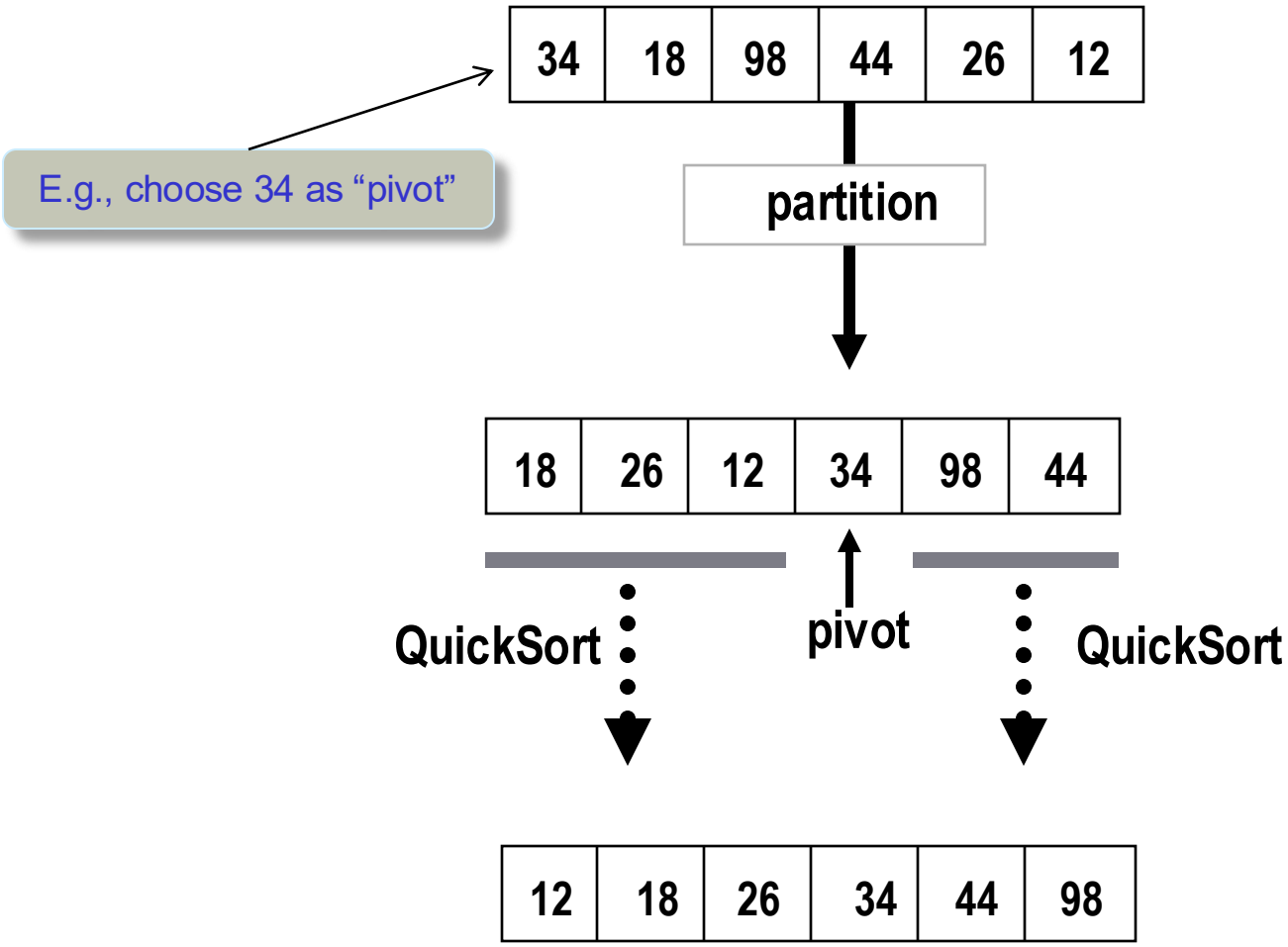
Quicksort Partitioning

Any element can be used as a pivot. For simplicity, we use number[low] as pivot **p**.



"pivot" **p** is now in its proper position

Quicksort Example



Quicksort Routine

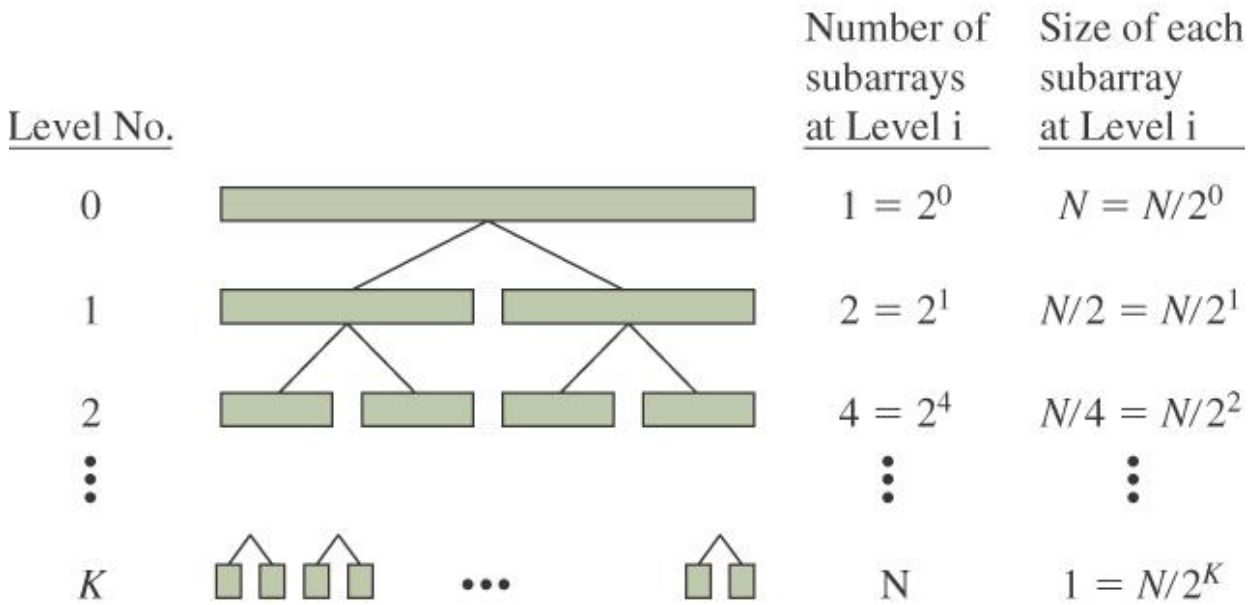
```
int[] quicksort(int number[], int length)
{
     int lesser[], greater[];
     int index;

     if (length <= 1)      // current array of length 0 or 1
          return index;    // so recurse back out

     // select a 'pivot' element from current array
     for (index = 0; index < length-1; index++) {
          if (int[index] < pivot)
               lesser.append(int[index]);   // build a lesser array
          else
               greater.append(int[index]);  // build a greater array
     }
     return concatenation(quicksort(lesser), pivot, quicksort(qreater));
} // end quicksort
```

This is the "conquer" step

This is the "divide" step

Quicksort Performance (1)

- In the worst case, Quicksort executes roughly the same number of comparisons as Selection and Bubble sorts: N^2
- On average, we can expect a partition process to split the array into two roughly equal sub-arrays.



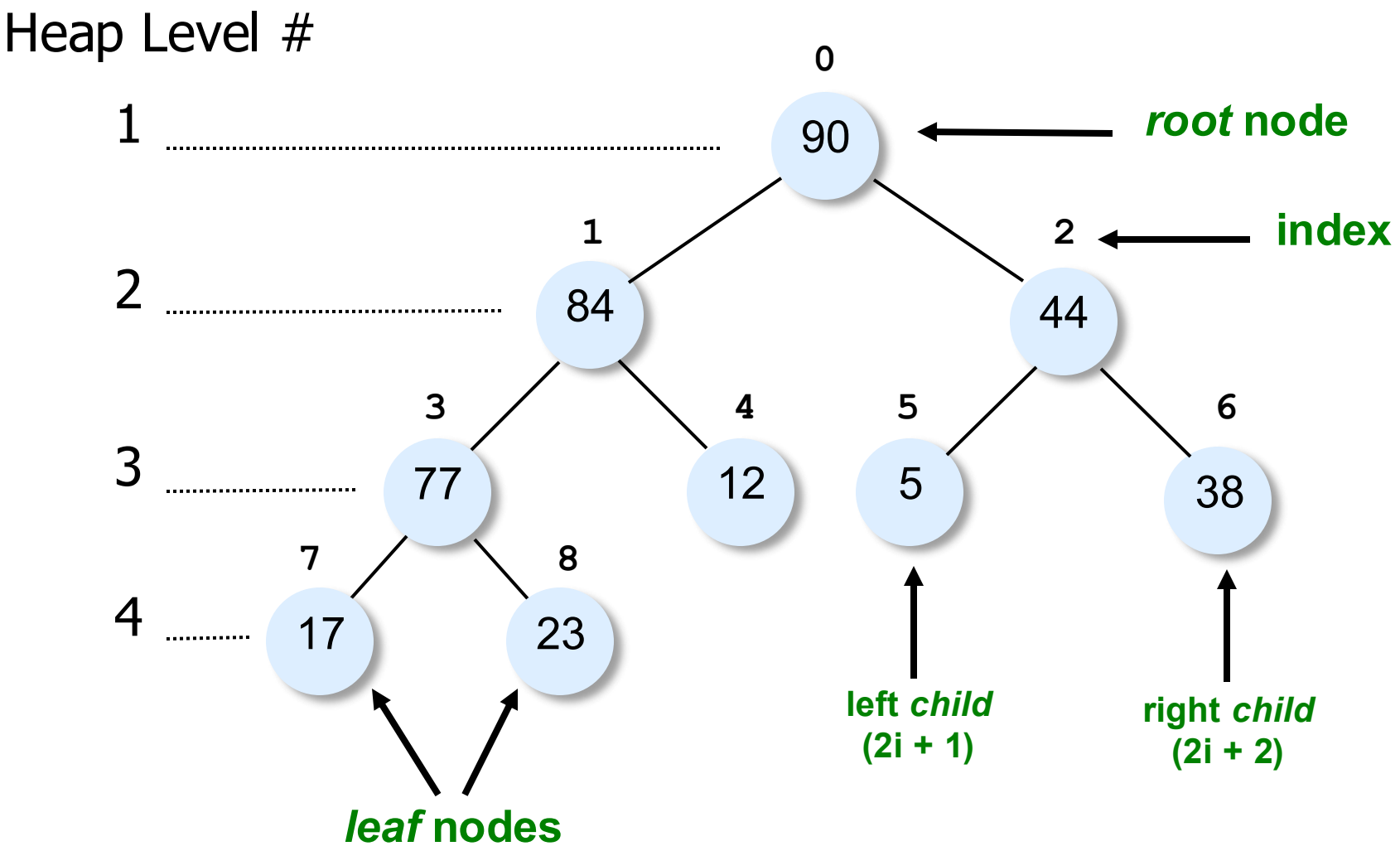
Quicksort Performance (2)

- There are K levels, for a total number of comparisons: $K*N$
- Since $1 = N/2^K$, solving for K we get: $N = 2^K$, and $K = \log_2 N$
- Therefore, on average (also the best-case) Quicksort performance is:
  $K*N$ (total comparisons) = $N \log_2 N$

Heapsort

- *Selection* and *Bubble* sorts are two fundamental sorting algorithms that take approximately N^2 comparisons to sort an array of N elements.
- The *Heapsort* algorithm improves the performance to approximately $1.5xNlog_2N$.
- The *Heapsort* algorithm uses a special data structure called a *heap.*
- A *heap* can be implemented very effectively using an array.

Heap Structure

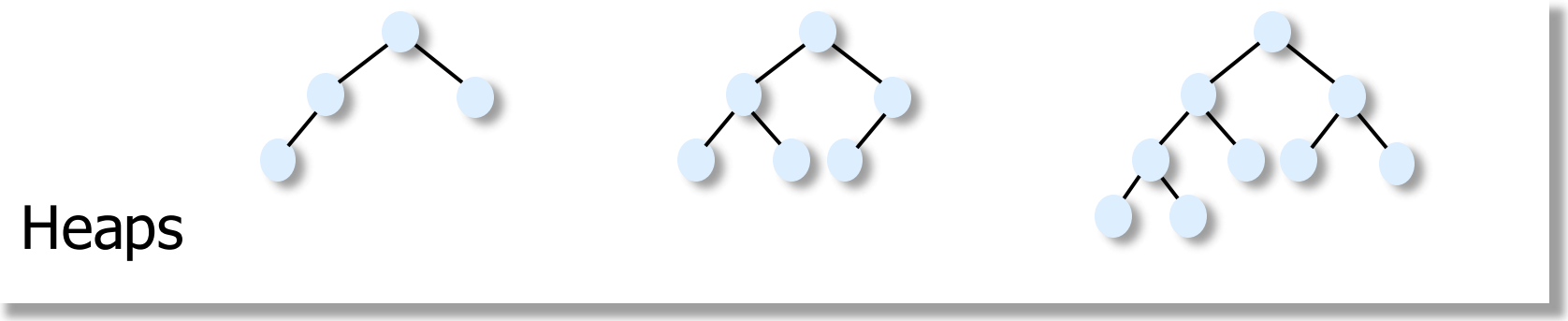


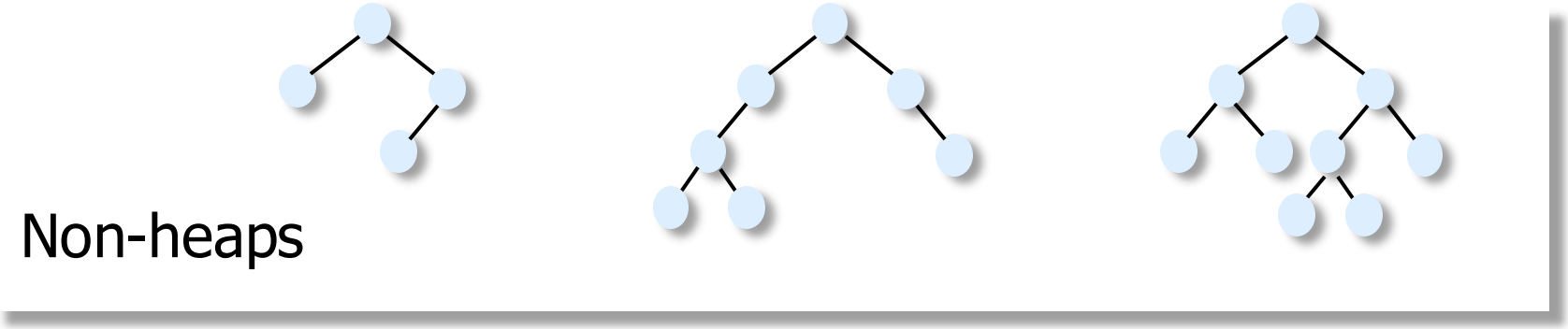
Heap Constraints

- A *heap* must satisfy the following two constraints (assume a *max-heap*):
  - *Structural Constraint*: nodes in a heap of size *N* must occupy the positions numbered 0, 1, ..., *N*-1, i.e., no open nodes allowed.
  - *Value Relationship Constraint*: a value stored in any node must be larger than the maximum of the values stored in its left and right *children*. Or, the value in any node must be less than its parent's value.

Structural Constraint

- Sample *heaps*, and *non-heaps* that violate the *structural constraint*:

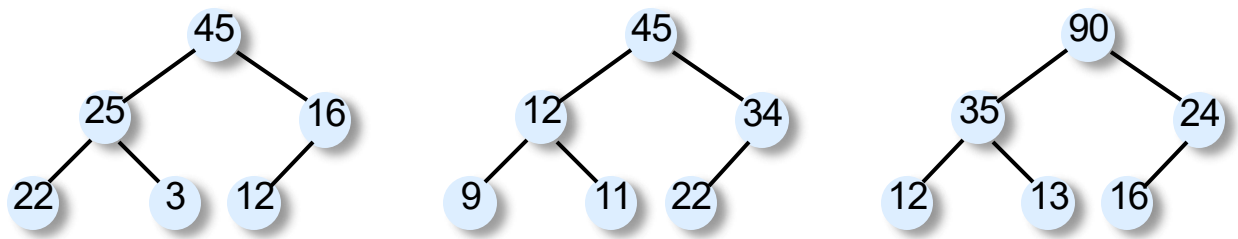




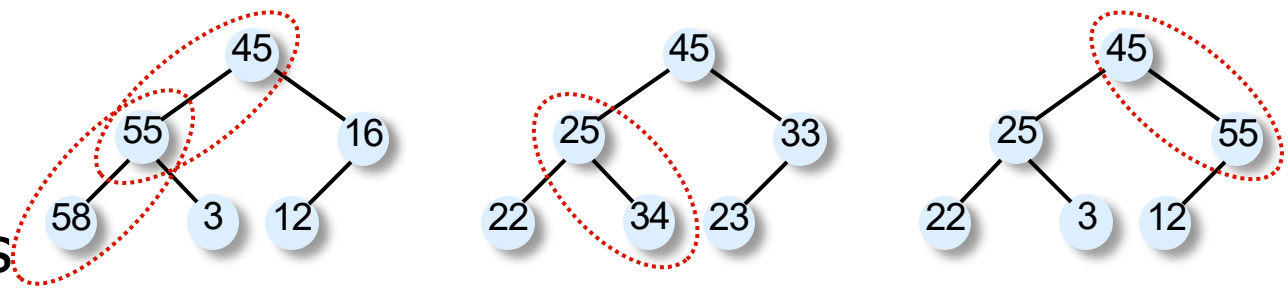
Value Relationship Constraint

- Sample *heaps* and *non-heaps* that violate the value *relationship constraint*:

Heaps



Non-heaps

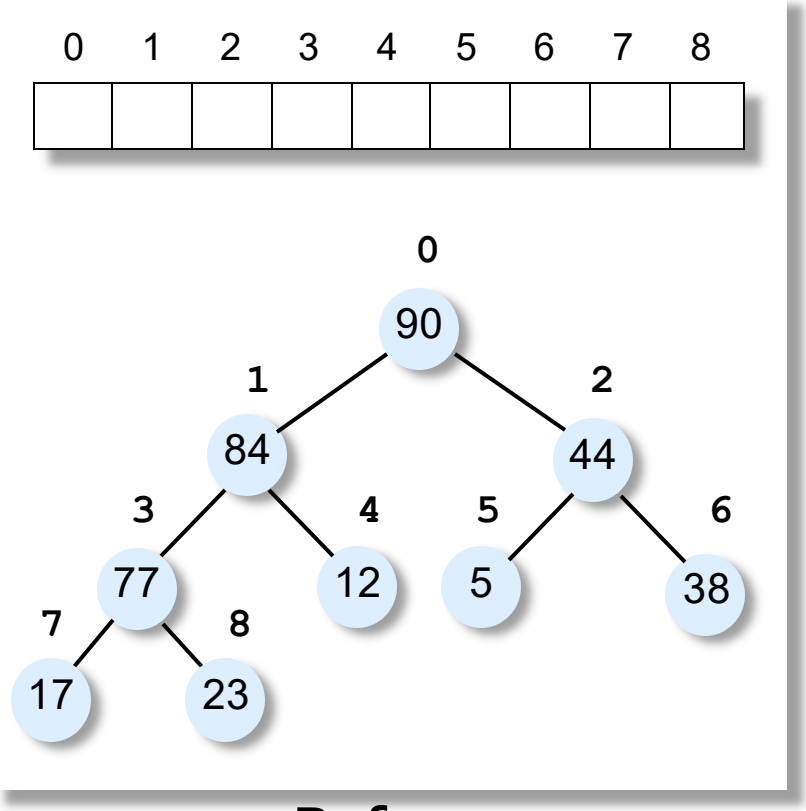


Heapsort Algorithm

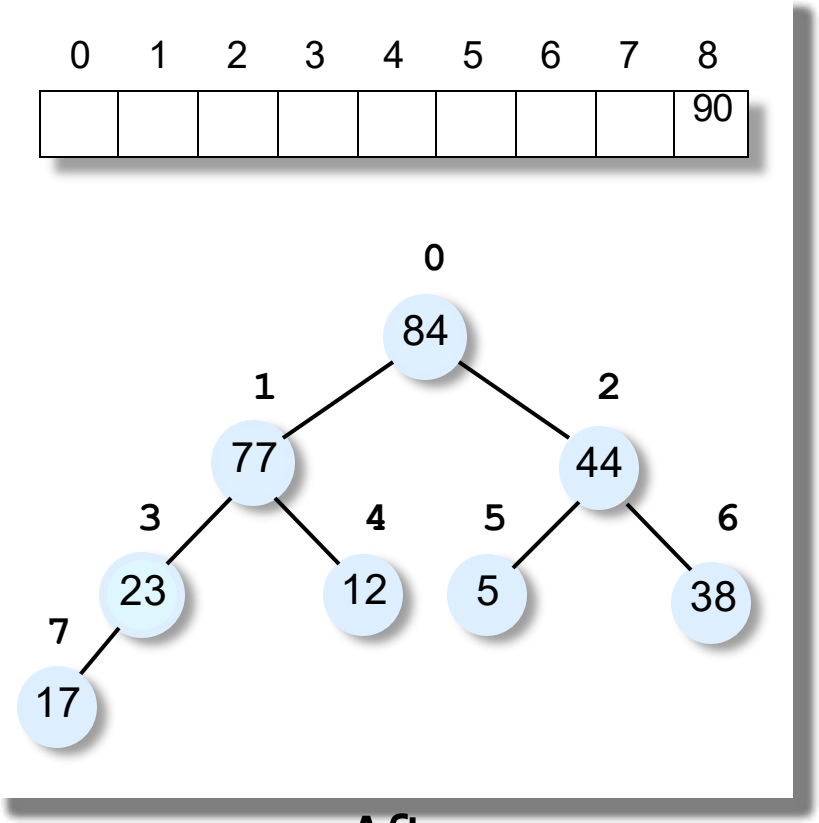
- How can we use a *heap* structure to sort *N* elements?
- *Heapsort* is carried out in two phases:
  - *Construction Phase*: construct a heap with given *N* elements.
  - *Extraction Phase*: pull out the value in the root successively, creating a new heap with one less element after each extraction.

Extraction Phase

- After each extraction, the heap must be rebuilt with one less element, e.g., one extraction step:



Before

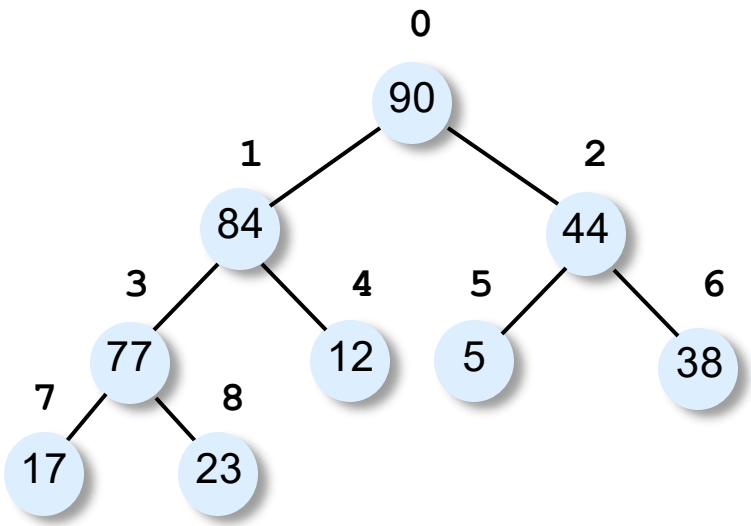


After

Rebuild Steps

- A sequence of rebuild steps to sort the array:

0	1	2	3	4	5	6	7	8



Before

0	1	2	3	4	5	6	7	8
5	12	17	23	38	44	77	84	90

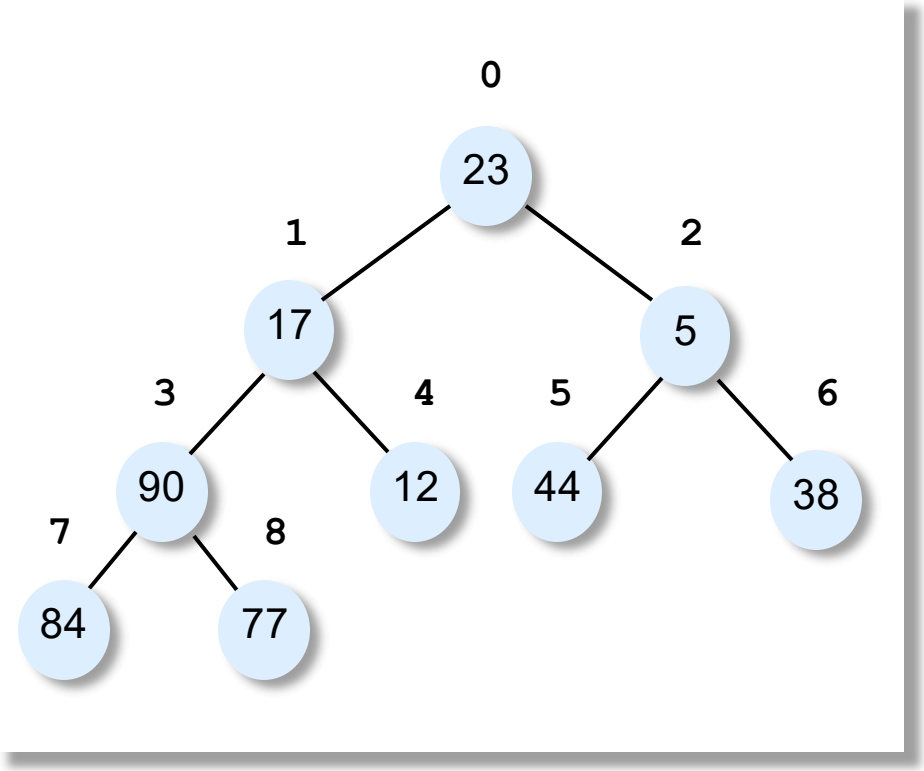
Sorting completed

after 8 rebuild steps

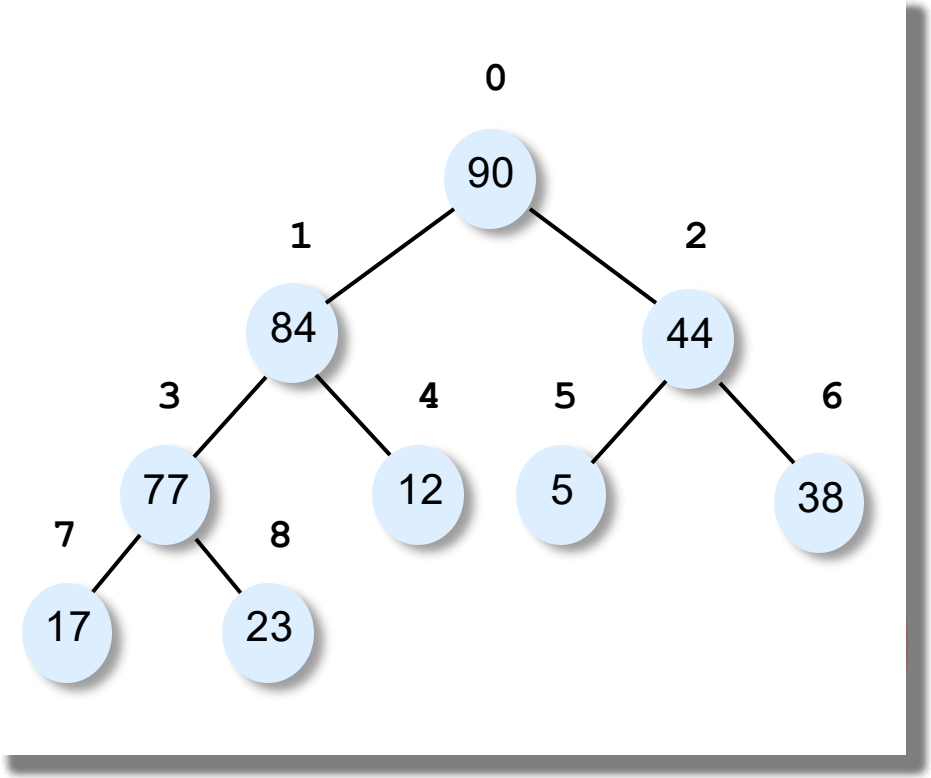
Done

Construction Phase

- Rebuild steps are bottom-up, starting from position $\lfloor (N-2)/2 \rfloor$ through position 0. *Why?*
- For an array of values: 23, 17, 5, 90, 12, 44, 38, 84, 77.



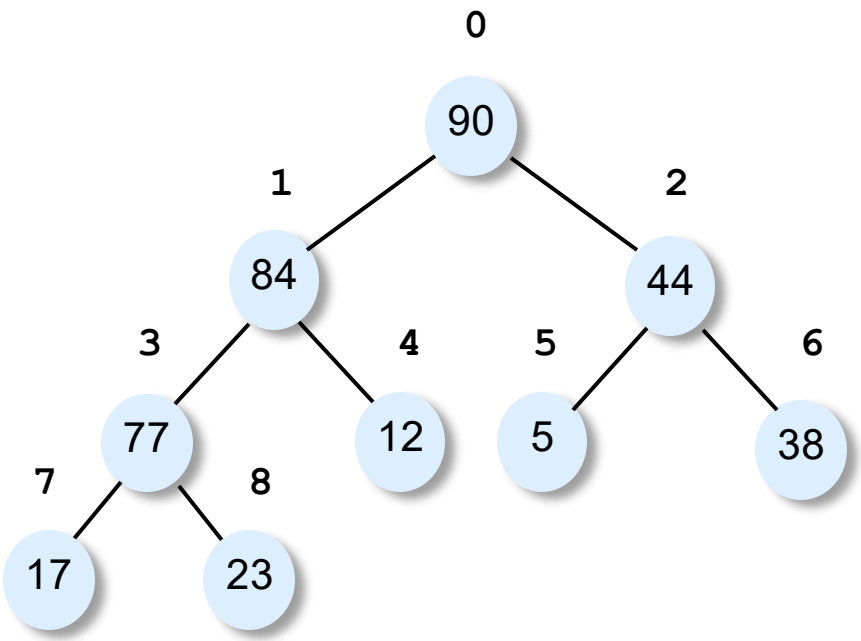
Before



Done

Heap Implementation

- We need to implement the abstract data structure heap into a concrete data structure:



Abstract Heap Structure

0	1	2	3	4	5	6	7	8
90	84	44	77	12	5	38	17	23

Notice this is a *heap*,

<u>not</u> a sorted array!

Concrete Array Implementation

The Extract Method to Sort

```
void heapExtract(int number[], int length)  // top-down algorithm
{
    for (int size = length-1; size >= 0; size--) {
        // extract root node; move last node up to be new root
        // rebuild the heap with one less element
        done = false;
        current = number[0];  // start at new root node
        while (!done) {
            if (current has no children) {
                done = true;
            } else {  // perform rebuild
                if (current node < larger child) {
                  // swap the two node values
                  // set current to the larger child
                } else
                  done = true;
            }
        }
    }
} // end heapExtract
```

The Construct Method

```
void heapConstruct(int number[], int length) // bottom-up algorithm
{
    for (int i = (length-2)/2; i >= 0; i--) {
        current = number[i];
        done    = false;
        while (!done) {
            if (current has no children) {
                done = true;
            } else {  // perform rebuild
                if (current node < larger child) {
                  // swap the two node values
                  // set current to the larger child
                } else
                  done = true;
            }
        }
    }
} // end heapConstruct
```

Heapsort Performance (1)

- In the extraction phase the total number of comparisons is (N-1)×K, where K is the depth of a heap.
- We solve for K using the fact that the heap must satisfy the structural constraint:

$$\sum_{i=1}^{K} 2^{i-1} = 2^K - 1$$

max # nodes in a heap with depth *K*

$$2^{K-1} - 1 < N \leq 2^K - 1$$

this holds because of the structural constraint

$$K = \lceil \log_2(N+1) \rceil$$

- Therefore...

$$(N-1) \times K \cong N \log_2 N$$

Heapsort Performance (2)

- In the construction phase, there are *N*/2 rebuild steps.
- Each rebuild step will take no more than *K* comparisons.
- The total number of comparisons in the construction phase is approximately...

$$\frac{N}{2}\log_2 N$$

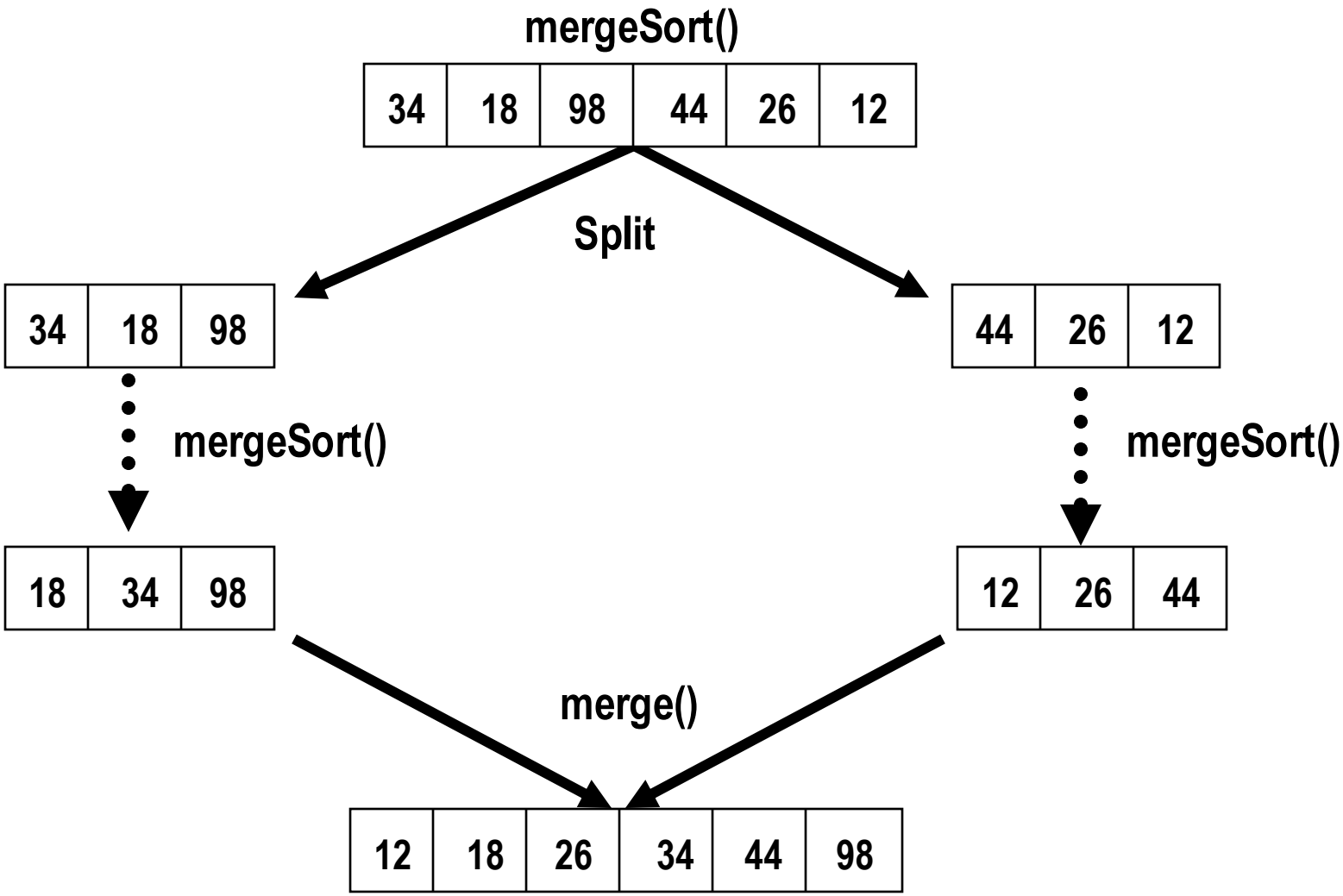
- Therefore, the total number of comparisons for both phases is...

$$\frac{N}{2}\log_2 N + N\log_2 N = \boxed{1.5N\log_2 N}$$

Merge Sort Algorithm

- Another recursive "*divide-and-conquer*" sorting algorithm:
  - Divide array of elements in (roughly) half, and then sort each half; continue halving until...
  - A sub-array of 0 or 1 elements is reached; that sub-array is now sorted, so *merge* this half with its other half. Continue merging halves until...
  - The original array halves are reached; *merge* these, and the original array is now sorted.

Merge Sort Algorithm



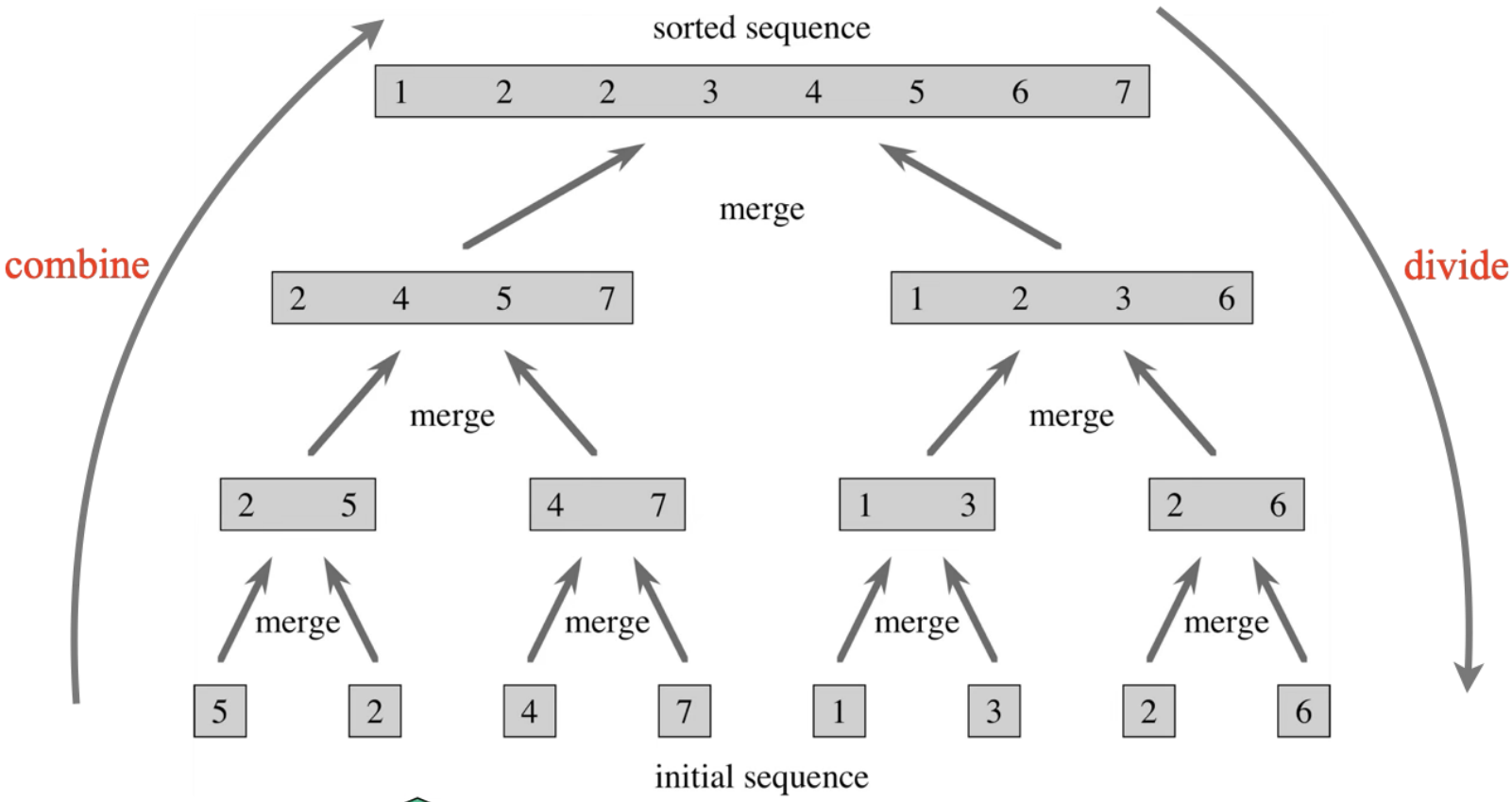
Merge Sort Routine (1)

```
void mergeSort(int number[], int low, int high)
{
    int mid;
    if (low < high) {  // 0-1 elements remaining?
        mid = (low + high)/2;   // integer division (floor)
        mergeSort(number, low, mid);     // recurse lower half
        mergeSort(number, mid+1, high);  // recurse upper half
        merge(number, low, mid, high);   // merge halves after
                                 // returning from recursive calls
    }
} // end mergeSort
```

- There are multiple ways to "*merge*" the halves:
  - The two halves are first copied into a temp array.
  - The temp array is scanned, and the next-greatest element successively copied back into the array.

Merge Sort Routine (2)

Example: MERGE-SORT(A, 1, 8) = [5, 2, 4, 7, 1, 3, 2, 6]



After repeated recursive calls to MERGE-SORT, we get 8 base-cases (1-element arrays, each sorted) ⇒ conquer

Merge Sort Performance

- Since we are not concerned with a “pivot” location (as in Quicksort), but instead always just halve the array, the average and worst cases are the same.
- The number of comparisons for both as calculated in the same way as Quicksort, is thus: $N \log_2 N$.

Questions?